Chicago Quant Alley Assignment 2 : Build a Quantitative Strategy Simulator from Scratch

Before Strategy Design: Foundations in Execution and P&L Modeling

Overview

This assignment will help you implement a discrete-time trading simulator to run basic option/futures strategies over historical data (collected from the Delta Exchange API or similar). The simulator must be modular, event-driven, and maintain its own trade book and P&L ledger.

You must write all logic from scratch. No usage of `backtrader`, `zipline`, `bt`, or similar libraries.

Directory Requirements

Your project should follow this structure:

```
/project_root/
 Simulator.py       # main simulation loop
 Strategy.py        # your custom strategy class
 utils/
    getStrikes.py  # utility for extracting strike prices
 stats/
    printStats.py  # post-simulation analytics (P&L, drawdown)
 data/
    20240601/
       MARK:BTCUSDT.csv
       MARK:C-BTC-70000-20240601.csv
       ...
 config.py          # user settings (startDate, endDate, symbols)
```

Part 1: Implement Your Simulator (40 pts)

You will implement the simulator in `Simulator.py`. Use the following method structure, and include each required block inside the method as described.

1. **Data Loader (10 pts)** — in `readData(self)`
   - Loop through all dates from `simStartDate` to `simEndDate` (inclusive).
   - For each date and each symbol from `config.symbols`, load the corresponding CSV file from the appropriate folder inside `data/`.
   - Add a column `"Symbol"` to each DataFrame to track source.
   - Concatenate all data into a single DataFrame `self.df`, then sort it by timestamp.
2. **Event Engine (10 pts)** — in `startSimulation(self)`

- Loop through each row of the sorted DataFrame.
- Extract the symbol and price from that row.
- Update the latest market price: `self.currentPrice[symbol]`.
- Pass the entire row to `self.strategy.onMarketData(row)`.

3. **Execution Engine (10 pts)** — in `onOrder(self, symbol, side, quantity, price)`
   - Simulate slippage:
     $$\text{Buy: price} \times (1+\epsilon) \quad \text{Sell: price} \times (1-\epsilon)$$
     where $\epsilon = 0.0001$.
   - Compute trade value = trade price × quantity.
   - Update:
     - `currQuantity[symbol]` (increment for buy, decrement for sell),
     - `buyValue[symbol]` and `sellValue[symbol]` accordingly.
   - Notify strategy: call `onTradeConfirmation(...)` with the adjusted trade price.
4. **P&L Engine (10 pts)** — in `printPnl(self)`
   - For each traded symbol, calculate:
     $$\text{P\&L} = (\text{Total Sell Value}) - (\text{Total Buy Value}) + (\text{Quantity} \times \text{Current Price})$$
   - Sum across all symbols.
   - Print or store the current total P&L.

Part 2: Implement Your Strategy Class (30 pts)

Create `Strategy.py` with a class named `Strategy`. Follow this structure:

```
class Strategy:
    def __init__(self, simulator): ...
    def onMarketData(self, row): ...
    def onTradeConfirmation(self, symbol, side, quantity, price): ...
```

- **In `__init__()`:**
  - Store reference to the simulator.
  - Initialize: entry time, entry price, strike prices, booleans for call/put position, total P&L.
- **In `onMarketData(row)`:**
  - Detect 1 PM on Day 1 using timestamp in `row['time']`.
  - At 1 PM, get ATM futures price and select closest put and call strikes (e.g., within 2%).
  - Sell 0.1 quantity of both (i.e., sell a straddle).
  - Track when to exit the position:
    - if futures price deviates $> 1\%$ from entry, OR
    - if cumulative P&L exceeds $+500$ or drops below 500.
  - On exit, buy back the options.
- **In `onTradeConfirmation(...)`:**
  - Update running P&L when an order is confirmed.
  - Keep a record of each trade's direction, symbol, price, and quantity.

Part 3: Analyze Simulation Output (30 pts)

Write a script (`printStats.py`) to analyze and visualize your results.

1. **Performance Metrics (15 pts)**
   - Compute:
     - Mean, median, and standard deviation of PnL.
     - Daily returns and Sharpe ratio.
     - Maximum drawdown using cumulative PnL.
     - Value at Risk (VaR) and Expected Shortfall (95%) from returns.
2. **Plots (15 pts)**
   - Plot:
     - Cumulative PnL curve.
     - Drawdown curve.
   - Export both plots as PNGs to include in your report.

Deliverables

- `Simulator.py`, `Strategy.py`, and `printStats.py`
- Output CSV of timestamped PnL
- PDF report (max 4 pages), including:
  - A short explanation of each module
  - Sample trades and plots
  - What worked, what didn't, and what you would improve

Bonus (+10 pts)

Use your simulator on real BTC options data from Delta Exchange for **5+ symbols** and present a multi-day simulation result.

This simulator is your base infrastructure for all future strategy work. Design carefully, and document clearly.